

Editorial Pedagógico: Prefixa, Infixa e Posfixa

Por: Sebastião Alves Brasil, UNIP

Data: Maio 2026

Skills: tree-traversal, recursion, strings

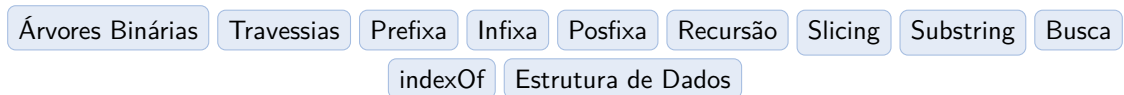
▷ Objetivo Pedagógico

- Compreender a relação intrínseca entre as três principais travessias de Árvores Binárias (Pré-Ordem, Em-Ordem, Pós-Ordem).
- Construir (ou simular a construção de) uma árvore a partir de dois de seus percursos originais.
- Aplicar recursão baseada na técnica de fatiamento de strings (*String Slicing*).
- Praticar manipulação eficiente de cadeias de caracteres.

1. Disciplinas Relacionadas

Disciplina	Tópico Específico
Estruturas de Dados	Árvores Binárias, Travessias em Árvores.
Projeto de Algoritmos	Divisão e Conquista (Recursividade).
Laboratório de Programação	Manipulação de Strings e Substrings.

2. Nuvem de Tópicos



3. Editorial Completo

3.1. As Ordens das Travessias

Travessias em Árvores Binárias definem como vamos "ler" ou "visitar" os nós da estrutura. Existem três travessias principais e seus nomes são derivados da posição em que a **Raiz (Nó atual)** é visitada em relação aos seus **Filhos (Esquerda e Direita)**.

- **Prefixa (*Pre-order*)**: (1º) Raiz, (2º) Esquerda, (3º) Direita.
- **Infixa (*In-order*)**: (1º) Esquerda, (2º) Raiz, (3º) Direita.
- **Posfixa (*Post-order*)**: (1º) Esquerda, (2º) Direita, (3º) Raiz.

O desafio do problema é obter a Posfixa, sendo que nos são dadas apenas a Prefixa e a Infixa.

3.2. O Segredo: A Intersecção das Travessias

Para remontarmos a árvore logicamente, nós precisamos saber qual nó é a Raiz e quais nós pertencem ao lado esquerdo ou direito dessa Raiz.

Observe as propriedades das strings fornecidas:

1. **Na string Prefixa:** O primeiro caractere da string é **sempre** a Raiz absoluta de toda aquela sub-árvore.
2. **Na string Infixa:** Se nós sabemos quem é a Raiz, procuramos ela na string Infixa. Ao encontrarmos, sabemos que **tudo à esquerda desta letra** pertence aos galhos esquerdos, e **tudo à direita desta letra** pertence aos galhos direitos!

Sabendo disso, podemos dividir o problema recursivamente. Em vez de criarmos objetos em memória representando a Árvore, podemos usar uma função que recebe duas strings e retorna uma string, simulando a montagem Pós-Fixa:

$$Posfixa = Resolver(Esq) + Resolver(Dir) + Raiz$$

3.3. Mapeando e Cortando as Strings

Para o algoritmo funcionar, precisamos cortar as strings originais e passar as partes corretas para a recursividade.

Passo a Passo:

1. Recebemos `pre` e `inf`. O tamanho inicial de ambas é o mesmo.
2. A Raiz é `root = pre[0]`.
3. Procuramos `root` dentro de `inf`, encontrando seu índice `pos`.
4. **Infixa Esquerda e Direita:**
 - O lado esquerdo é a substring do índice 0 ao índice `pos-1`.
 - O lado direito é a substring do índice `pos+1` ao final da string.
5. **Prefixa Esquerda e Direita:**
 - Note que os nós da esquerda possuem o mesmo tamanho! O tamanho do infixo esquerdo é exatamente `pos`. Logo, o lado esquerdo da prefixa vai do índice 1 (pulando a raiz que estava no 0) até `pos` caracteres para frente.
 - O restante (a partir do índice `pos+1`) formará a prefixa direita.
6. Chamamos recursivamente a função para as strings da Esquerda e, em seguida, as strings da Direita, e acrescentamos o `root` ao final da resposta resultante.

3.4. Trace de Execução do Exemplo 3

Exemplo: `pre = ABCDEF`, `inf = CBAEDF`

Nível 1: - Raiz = 'A' - Índice de 'A' na Infixa (pos): 2. - Infixa Esq: CB. Infixa Dir: EDF. - Prefixa Esq: BC. Prefixa Dir: DEF. - Ação: Vai pedir $Resolver(BC, CB)$ e depois $Resolver(DEF, EDF)$. No fim vai colocar 'A'.

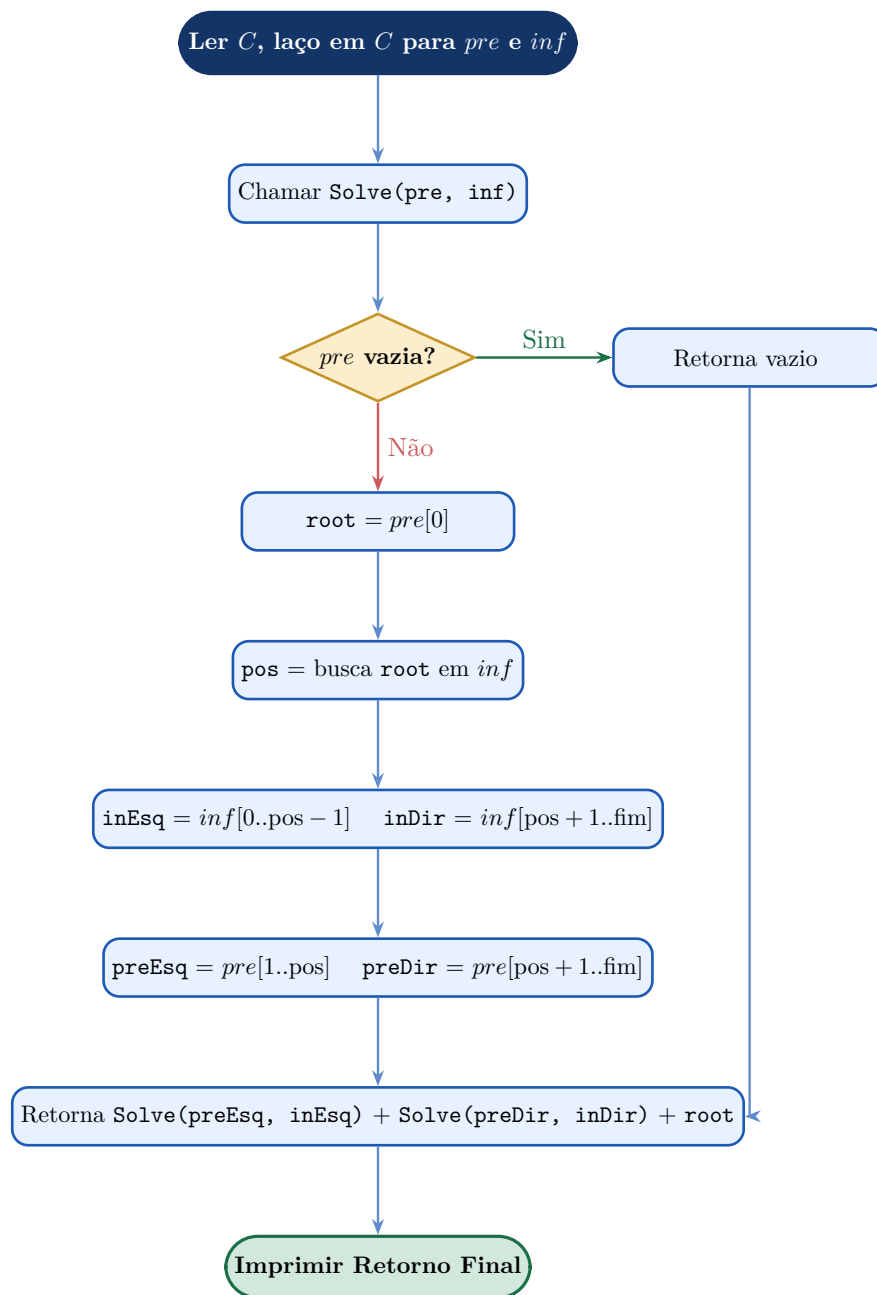
Nível 2 (Filho Esquerdo): $Resolver(BC, CB)$ - Raiz = 'B'. - Índice de 'B' em CB (pos): 1. - Infixa Esq: C. Infixa Dir: vazio. - Prefixa Esq: C. Prefixa Dir: vazio. - Resposta disto retornará: $Resolver(C, C)$ + vazio + 'B'.

No limite da recursão $Resolver(C, C)$ retorna 'C'. Logo este bloco retorna 'CB'.

Nível 2 (Filho Direito): $Resolver(DEF, EDF)$ - Raiz = 'D'. - Índice de 'D' em EDF (pos): 1. - Infixa Esq: E. Infixa Dir: F. - Retorna 'E' + 'F' + 'D' = 'EFD'.

Retornando ao Nível 1: - Resposta Total = $Esq + Dir + Raiz$ = 'CB' + 'EFD' + 'A' = **CBEFDA!**

4. Fluxograma da Solução



5. Análise de Complexidade

Etapa	Descrição	Complexidade
Busca do Índice	A função <code>indexOf</code> (ou equivalente) percorre o array Infixo até achar a Raiz.	$\mathcal{O}(N)$
Fatiamento (Slicing)	A criação de novas strings a cada recursão copia caracteres proporcionais ao tamanho do lado do galho.	$\mathcal{O}(N)$
Recursão	Nos piores casos (árvores "linhas retas"perfeitamente desbalanceadas), a recursão vai à profundidade N , fazendo uma busca $\mathcal{O}(N)$ em cada nó.	$\mathcal{O}(N^2)$
Total	Como as strings têm tamanho máximo de $N = 52$, o tempo computacional para $\mathcal{O}(N^2)$ por caso é cerca de 2700 operações básicas, infinitamente seguro.	$\mathcal{O}(N^2)$
Espaço	Espaço das copias de strings (pilha de chamadas recursivas).	$\mathcal{O}(N^2)$

6. Tutorial Recomendado

Referências Bibliográficas

- **Algoritmos: Teoria e Prática (Cormen et al.)** — Capítulo 12: Árvores Binárias de Busca (Percorrendo a árvore).
- **Competitive Programming 3** — S. Halim. Representação de Árvores implícitas via string / arrays paralelos.

Editorial pedagógico gerado para o Mundo do Código — Pack Técnicas. ID 1194. Autor do Problema: Sebastião Alves Brasil.